

Cluster and Conquer: Malicious Traffic Classification at the Edge

Alec F. Diallo¹ and Paul Patras², *Senior Member, IEEE*

Abstract—The uptake of digital services and IoT technology gives rise to increasingly diverse cyber attacks, with which commonly-used rule-based Network Intrusion Detection Systems (NIDSs) struggle to cope. Therefore, Artificial Intelligence (AI) supports a second line of defense, since this methodology helps in extracting non-obvious patterns from network traffic and subsequently in detecting more confidently new types of threats. Cybersecurity is however an arms race and intelligent solutions face renewed challenges as attacks evolve while network traffic volumes surge. We propose Adaptive Clustering-based Intrusion Detection (ACID), a novel approach to malicious traffic classification and a valid candidate for deployment at the network edge. ACID addresses the critical challenge of sensitivity to subtle changes in traffic features, which routinely leads to misclassification. We circumvent this problem by relying on low-dimensional embeddings learned with a lightweight neural model comprising multiple kernel networks that we introduce, which optimally separates samples of different classes. Extensive experiments with datasets spanning 20 years demonstrate ACID attains 100% accuracy and F1-score, and 0% false alarm rate, significantly outperforming state-of-the-art clustering methods and NIDSs. Furthermore, our results show that ACID offers a high degree of robustness to input perturbations, while intrinsically providing a framework for continual learning.

Index Terms—Network intrusion detection, kernel-based clustering, deep learning, continual learning.

I. INTRODUCTION

THE ADOPTION of Internet of Things (IoT) devices and cloud-based services continues to grow sharply [2], leading to a pressing need for robust and efficient defense mechanisms to safeguard the networking infrastructure and users' private data. This is particularly critical as attackers continue to discover new system/software vulnerabilities on a daily basis [3]. As a result, cybercrime costed businesses and individuals in the United States alone \$3.5 billion in 2019 [4]. Meanwhile, traditional security measures such as firewalls, anti-viruses, and rule-based Network Intrusion Detection Systems (NIDSs) are unable to keep up with the most recent and sophisticated attacks that exploit loopholes

to bypass the perimeter defenses set by these measures [5]. In particular, widely-deployed NIDSs, including Snort [6], Zeek [7], or Suricata [8] present a number of disadvantages. Namely, they (i) require frequent updates of signature databases; (ii) exhibit high false alarm rates when classifying traffic with evolving behavior; and (iii) depend on considerable levels of human expert intervention for system tuning and manual decision making.

In this context, Artificial Intelligence (AI)- and Machine Learning (ML)-based techniques such as Artificial Neural Networks, Clustering, and Ensemble Learning are increasingly appealing for building automatic network threat or anomaly detection systems [9], [10]. This is largely due to the unique ability of neural models to discover hidden patterns in vast amounts of data, which helps boosting classification accuracy, as already demonstrated in several research areas including speech recognition [11], computer vision [12], and wireless and mobile networking [13].

However, despite the rapid progress of AI-based approaches to Network Intrusion Detection (NID), existing solutions (e.g., [14], [15], [16]) remain extremely sensitive to small changes in individual features of network traffic flows, which dilutes their effectiveness in the face of continuous software updates and evolving traffic landscapes, as we reveal. Specifically, since these techniques learn from features of individual samples, training them on small subsets of carefully crafted features, unwittingly mislabelled samples, or unbalanced datasets negatively impacts their generalization abilities, thereby rendering the detection of new malicious network activity very difficult. Additionally, current NIDSs introduce application latency due to their complexity, while their architectures are usually fixed, thus requiring retraining for every new task.

To tackle these problems, in this paper, we propose ACID, a classifier-agnostic and highly-effective Adaptive Clustering-based Intrusion Detection system, highly suited for deployment on resource-constrained devices at the network edge. Our design incorporates an original multi-kernel based neural network that enables our NIDS to generalize well, regardless of any small changes incurred by small groups of packets or the unbalanced nature of the training dataset. We achieve this by means of a clustering algorithm that learns low-dimensional embeddings from linear and non-linear combinations of network flow features, which makes it possible to unambiguously separate these flows. By combining the cluster centers learned through our clustering network with statistical and semantic features extracted from packet

Manuscript received 30 May 2023; revised 28 September 2023; accepted 3 December 2023. Date of publication 13 December 2023; date of current version 12 July 2024. This work was supported by Arm Ltd. A preliminary version of this paper was published at IEEE Conference on Computer Communications 2021 (INFOCOM'21) [1]. The associate editor coordinating the review of this article and approving it for publication was N. Zincir-Heywood. (Corresponding author: Alec F. Diallo.)

The authors are with the School of Informatics, The University of Edinburgh, EH8 9AB Edinburgh, U.K. (e-mail: alec.frenn@ed.ac.uk; paul.patras@ed.ac.uk).

Digital Object Identifier 10.1109/TNSM.2023.3342716

sequences, and feeding the resulting feature vectors to a classifier, we effectively improve the detection performance of the NIDS while minimizing the false alarm rate. In light of the practical challenges faced by current NIDS solutions, there is a growing demand for advanced defense mechanisms that can adapt to the evolving threat landscape. Addressing issues such as dealing with increasing volume and complexity of network traffic, designing efficient and adaptive algorithms, detecting threats with minimal human intervention, and guaranteeing reduced false alarm rates is therefore crucial for ensuring the effective deployment and operation of NIDS. To this end, our proposed solution provides a reliable and efficient framework for building defenses against emerging threats in the digital domain. In a nutshell, our **key contributions** can be summarized as follows:

- [C1] We introduce a novel supervised Adaptive Clustering (AC) technique that learns optimal low-dimensional embeddings of complex data, enabling the enhancement of dataset features with representative cluster centers to improve the robustness and generalization of classification models, particularly for NID problems.
- [C2] Building on our clustering approach, we design a NIDS that extracts features from raw packets, which when extended with learned cluster centers allow effective and reliable binary and multi-label classification.
- [C3] We demonstrate the effectiveness of our solution, ACID, on six different network traffic datasets containing illicit flows spanning 20 years, and reveal its potential for complete and accurate detection of all known types of threats, surpassing existing NID approaches by a substantial margin of up to 47% in F_1 -score.
- [C4] We assess the computational efficiency and runtime requirements of ACID, showing its suitability for deployment on resource-constrained edge devices, and validate its robustness in practical deployments, underscoring its capacity to withstand noisy environments, while highlighting its unique and inherent ability to enhance NIDS through continual learning for increased resilience and adaptability in the dynamic realm of cyber threats.

To the best of our knowledge, ACID is the first Deep Learning (DL)-based NIDS that exploits adaptive clustering to minimize false alarm rates, it is robust to a range of malicious traffic types, and it is amenable to prototyping on commodity gateways for real-time threat detection at the network edge.

II. RELATED WORK

We briefly overview NID approaches related to our ACID system, with a focus on deep learning and clustering based approaches and their limitations. We also discuss the current state of continual learning research, specifically from the standpoint of model resilience and adaptability, and highlight the shortcomings that our proposed solution aims to address.

A. Deep Learning-Based Intrusion Detection

Most DL-based NIDSs attempt to match observed network flows against previously learned patterns. Despite increasing

adoption, they produce unacceptably high false alarm rates for relatively small gains in detection performance. This significantly limits their applicability to real-life scenarios. Auto-Encoders (AEs) can learn latent representations of features and reduce their dimensionality in order to minimize memory consumption, which motivates their use for anomalous traffic detection [20], [21], [22]. Tan et al. apply Convolutional Neural Networks (CNNs) to learn spatial representations of packets, followed by image classification methods to identify malware traffic [23]. Wang et al. combine CNNs and Long Short-Term Memory (LSTM) structures to learn both spatial and temporal correlations between features [24]. Despite the effectiveness of these techniques, they completely ignore time-based statistical features that can be inferred from packets and the semantic relationships within packet payloads. Min et al. use these ignored attributes and apply Natural Language Processing techniques to process packet payloads [25]. This boosts detection performances, yet still presents several important weaknesses, including ignoring dataset imbalance and exhibiting very high processing times when dealing with large datasets. Under- and oversampling methods [26], [27] can mitigate this class imbalance problem, but these techniques either reduce the number of training data samples or use additional artificially generated data, both of which negatively impact classification performance, as they restrain the ability of ML models to learn accurate representations.

B. Clustering Based Intrusion Detection

Ideally, any Intrusion Detection System (IDS) should (i) have learning and hierarchical feature representation abilities; (ii) handle high-dimensional data and extract valuable patterns. Since clustering methods group data into meaningful sub-classes, seeking to separate members of different clusters, several IDSs build on this approach. Jianliang et al. use k-means clustering to detect unknown attacks and separate large data spaces effectively [28]. However, their approach suffers from degeneracy and cluster dependence, which could be overcome with the Y-means clustering algorithm proposed in [29]. Mingqiang et al. introduce the concept of graph-based clustering for anomaly detection, whereby a Local Deviation Coefficient Graph Based (LDCGB) approach identifies outliers [30]. Li et al. use a Particle Swarm Optimization (PSO) algorithm based on swarm intelligence [31]. This solution avoids falling into local minima, while providing good overall convergence. Multi-stage techniques improve NID by (i) generating meta-alerts through clustering and (ii) reducing false alarm rates via classification of these meta-alerts [32].

However, these approaches are frequently unable to discriminate superficially similar but in essence different attacks (e.g., U2R and R2L vs. benign), present high misclassification rates due to their unsupervised nature, and/or are computationally expensive, making them unfit for deployment on constrained devices. We overcome these issues through a simple and effective adaptive clustering approach, while offering significant improvements in detection rate and minimizing false alarms.

C. Continual-Learning

In recent years, there has been a growing interest in developing deep learning models that exhibit robustness and adaptability in network intrusion detection systems (NIDS). Several studies have explored techniques to improve the resilience of deep learning models to random noise and enhance their support for continual learning. For instance, Goodfellow et al. [63] investigated catastrophic forgetting in gradient-based neural networks, aiming to develop preventative methods and ensure efficient adaptation to new tasks as they are encountered. Von Oswald et al. [64] proposed the use of hypernetworks for continual learning, creating dedicated sub-networks to handle new tasks without retraining the entire model. Draelos et al. [65] introduced a neurogenesis deep learning approach to extend deep networks to accommodate new classes, while Rusu et al. [66] presented progressive neural networks that integrate new knowledge without forgetting the old. Furthermore, Finn et al. [67] proposed a model-agnostic meta-learning technique for fast adaptation of deep networks. In contrast to our approach, existing work still face many issues such as catastrophic forgetting in gradient-based neural networks, increased model complexity, scalability issues, or dependency on initial models.

III. THREAT MODEL

We consider both home networks and enterprise environments subject to two offensive scenarios. First, we envisage attackers located outside the target network, to which they attempt to gain access, compromise victim devices, or retrieve sensitive data. In this scenario, the attacker would scan all the devices connected to the network to find weaknesses that would allow access. In the second scenario we consider, the attacker is either located outside the target network, which was already compromised (giving them the ability to control the hijacked hosts), or is internally connected to that network. In both cases, we assume an NIDS is deployed on an edge device, where all incoming and outgoing network traffic packets can be captured. Furthermore, we consider the possibility where an attacker has acquired sufficient knowledge of the network infrastructure, including target IP addresses, open port numbers, etc. However, we assume the deployed NIDS is hardened and attackers do not have the ability to access or alter its behaviour.

Our proposed system is best suited to networks where most user traffic belongs to a finite set of known applications. Therefore, as we monitor all incoming and outgoing communications, we are able to train our neural model in a supervised manner on a labelled dataset.

IV. SYSTEM ARCHITECTURE

We propose a novel Deep Learning (DL)-based NIDS that maximizes the probability of detecting malicious network flows and minimizes the false alarm rate. Our approach aims to (i) quickly adapt to complex data structures and patterns, by discovering low-dimensional embeddings of networks flows, which optimally separate samples of different types; and (ii) be suitable for deployment on devices with limited computational capabilities situated at the periphery of the network, enabling

efficient threat detection security breaches, while enhancing real-time response capabilities and reducing the burden on centralized security infrastructures. These are essential features any NIDS must meet to be suitable for practical real-life environments. We fulfill these goals by combining three key components shown in the high-level overview of our ACID system in Fig. 1, namely:

- *Feature Extractor Module*: transforms raw network packets into vectors of header and statistical features, and (optionally) semantic representations of payloads (i.e., additional feature vectors);
- *Adaptive Clustering Module*: builds low-dimensional embeddings of network flow features and computes a set of abstract attributes that are common to samples belonging to the same traffic type;
- *Classification Module*: uses features extracted from both network flows and by the clustering module to improve detection rate and minimize the impact of outliers. This module corrects any misclassifications made via clustering and can exploit further correlations in the inputs.

In what follows, we detail the operation of each of these modules, then demonstrate how the synergies among them lead to remarkable malicious traffic detection performance.

A. Feature Extractor

ACID handles very large amounts of traffic by processing streams of raw network packets into feature-based representations of bidirectional flows corresponding to communications between (source, destination) pairs, over specific applications or protocols. These are subsequently used for clustering and classification. Our feature extractor comprises two parallel processing pipelines: (1) a *header analyzer* logic that builds a set of header and statistical features (including source/destination port numbers, packet inter-arrival time, total number of packets in a flow, etc.), which provide a compact representation of traffic behaviors; and (2) an optional *word embedding* logic that builds vectors of semantic representation of the payloads, through word2vec [33] and Text-CNN [34] techniques, similarly to the methodology previously used in [25]. While this additional payload features extractor unit significantly increases the computation costs, it can improve the performance of the classifier. This is because the semantic representations of payload features encode important information about contents of payload-based attacks, such as in the case of SQL injection.

Depending on the computational power of the device where the NIDS is deployed, the feature extraction module may introduce some latency. Regardless, online operation is easily achievable if running the NIDS separately from the packet forwarding unit. It is also worth noting that by aggregating packets into bidirectional flows, we drastically reduce the size of the training and evaluation sets, which in turn reduces the inference time of ACID.

B. Adaptive Clustering Module

One of our key contributions is a novel technique that improves the generalization abilities and robustness of any DL-based classification engine. In essence, we propose a

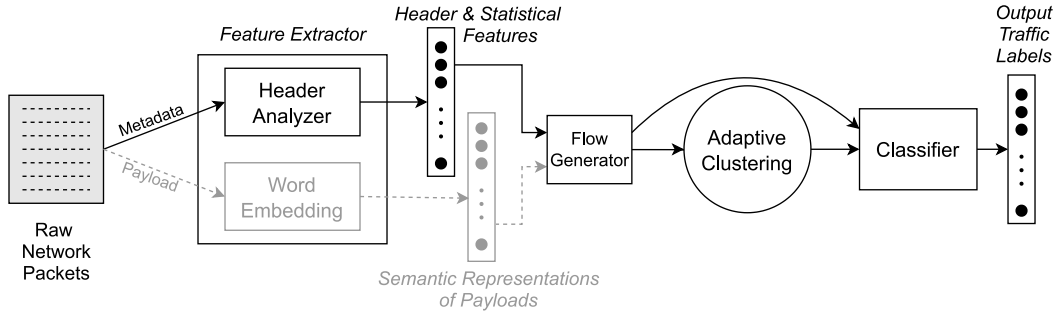


Fig. 1. Overview of the proposed ACID system. Header and statistical features, and (optionally) payload features are extracted from raw network packets, then grouped into bidirectional flows. Our AC module takes these flows, computes cluster centers, and appends them to the extracted features, before feeding the resulting vectors to a classifier for final decisions.

clustering algorithm that produces cluster centers to be used as extensions of the input features being clustered. As our Adaptive Clustering (AC) approach is designed to be end-to-end differentiable, the training is performed on mini-batches, whereby the network learns low-dimensional representations of the inputs and computes the corresponding kernel centers. This operation is performed online (in an iterative manner) and the final layer of the kernel networks yields the probabilities of each sample in the inputs belonging to all possible classes. For our NIDS, we retrieve the computed cluster centers of each class and use them to expand their samples' features, thereby obtaining more features for each data point. These combined features are then given to the classification module for final decisions about the nature of the traffic observed.

Since we aim to deploy our NIDS in a live environment, we aim at a clustering algorithm which:

- *Handles very large amounts of high-dimensional data points* – This requirement prevents us from using clustering approaches that need all training data at once.
- *Handles incoming streams of data at random time intervals* – New incoming data should not require retraining the entire clustering algorithm; instead, data distributions should be learned on the fly.
- *Quickly and effectively clusters even the most complex data points in multi-dimensional spaces* – Having semantically good clusters would boost the performance of the classifier, as proven by our experiments.
- *Produces cluster centers* – This is an essential requirement for improving the robustness and generalization abilities of the classifier.

These set of desired properties lead to designing a *plug & play* type DL-based clustering algorithm that can be trained with batches of data at a time, which we detail in Section V.

C. Classification Module

Finally, ACID employs a classification module that processes the combined extracted features and cluster centers for each sample, and outputs the inferred traffic class to which that flow belongs. While the classifier's architecture can be designed to obtain any desired property (e.g., exploit spatio-temporal correlations or perform binary/multi-label classification), the additional features provided by our AC algorithm improve classification performance regardless

of the specifics of the classifier, as they reduce the divergence between samples of the same class. To further reduce this divergence, we use each sample's corresponding cluster center instead of its low-dimensional representation, when extending its features vector. This not only improves the accuracy of the NIDS, but also minimizes the impact of outliers. To support this claim, we provide a theoretical proof of the gain introduced by using cluster centers as additional features.

Theorem 1 (Clustering-Based Divergence Reduction): Let K_c be the set of features obtained from the cluster center of a given class C . For any two samples $\chi_i, \chi_j \in C$,

$$\text{distance}(\chi_i \cup K_c, \chi_j \cup K_c) \leq \text{distance}(\chi_i, \chi_j),$$

Proof: Let χ_i and χ_j be n -dimensional real-valued vectors:

$$\begin{aligned} \chi_i &= \{x_{i1}, x_{i2}, \dots, x_{in}\} \in \mathbb{R}^n, \\ \chi_j &= \{x_{j1}, x_{j2}, \dots, x_{jn}\} \in \mathbb{R}^n, \end{aligned}$$

where x_{it} and x_{jt} correspond to individual features of the sample data, with $t \in \{0, 1, \dots, n\}$. Assume that χ_i and χ_j both belong to the C -th cluster according to our clustering module, and the C -th cluster center is defined as

$$K_c = \{k_{c1}, k_{c2}, \dots, k_{cm}\} \in \mathbb{R}^m.$$

The aggregated features obtained after clustering are then

$$\begin{aligned} \chi'_i &= \{x_{i1}, x_{i2}, \dots, x_{in}, k_{c1}, k_{c2}, \dots, k_{cm}\} \in \mathbb{R}^{n+m}, \\ \chi'_j &= \{x_{j1}, x_{j2}, \dots, x_{jn}, k_{c1}, k_{c2}, \dots, k_{cm}\} \in \mathbb{R}^{n+m}. \end{aligned}$$

An intuitive way to calculate the impact of the clustering on robustness is to compare the similarities between the original and aggregated feature vectors, i.e., $Q_1 = \text{distance}(\chi_i, \chi_j)$ vs $Q_2 = \text{distance}(\chi'_i, \chi'_j)$. Expanding each yields

$$\begin{aligned} Q_1 &= \frac{1}{n} \sum_{\alpha=1}^n (x_{i\alpha} - x_{j\alpha})^2, \\ Q_2 &= \frac{1}{n+m} \left(\sum_{\alpha=1}^n (x_{i\alpha} - x_{j\alpha})^2 + \sum_{\alpha=1}^m (k_{c\alpha} - k_{c\alpha})^2 \right) \\ &= \frac{1}{n+m} \sum_{\alpha=1}^n (x_{i\alpha} - x_{j\alpha})^2. \\ Q_1 - Q_2 &= \frac{m}{n+m} \cdot Q_1 \implies Q_2 = \beta \cdot Q_1, \end{aligned}$$

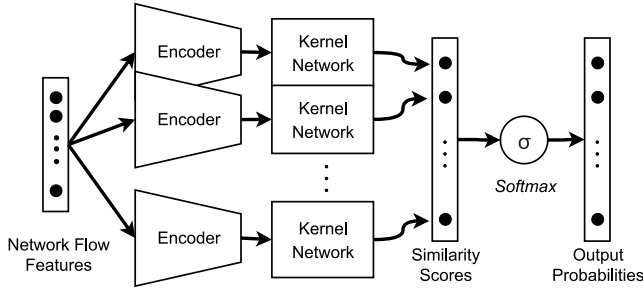


Fig. 2. Architecture of our Adaptive Clustering network.

where $\beta = n/(n + m) \leq 1, \forall n > 0, \forall m \geq 0$, is the impact measure obtained by using the cluster centers as additional features. This implies that $Q_2 \leq Q_1, \forall m \geq 0$. ■

Hence, using cluster centers as additional features can only improve the decision capability of the classifiers by reducing the differences between items of the same group. The experimental results we present in Section VII fully support this finding. In our experiments, we will use a Random Forest structure [35] to perform the final classification of traffic flows.

It is worth noting that our clustering approach is not limited to NID, but can be applied to any other classification task.

V. ADAPTIVE CLUSTERING

Clustering algorithms learn some notion of similarity within a dataset, to group similar samples together. They usually heavily depend on individual data points and (i) bear significant time complexity when handling large amounts of data or large sample dimensionality; (ii) require an explicit measure of “distance” (even in multi-dimensional spaces); and (iii) return multiple possible interpretations of clustering results. Neural Networks (NNs) tackle these issues, yet often at the cost of reduced clustering performance. This is particularly problematic when dealing with complex datasets, where the NN, in seeking to generalize to all possible clusters (or classes), ends up only able to correctly classify a small subset of unambiguous samples. Unfortunately, most realistic datasets are multi-dimensional, wherein there are no obvious distinct patterns between members of different classes.

To tackle this problem, the Adaptive Clustering (AC) method we propose builds on multiple kernel networks, each learning to tailor itself to one of the possible clusters associated with a particular type of traffic which we want to classify, as illustrated in Fig. 2. For any observed sample, different encoders discover its optimal low-dimensional embedding, and kernel networks learn to extrapolate a general representation (i.e., cluster center) of its group’s members. By using encoders, our model reduces the dimensionality of the input features to any desired dimension. While the encoder architecture can be selected according to the task at hand, using a simple multi-layer, feed-forward NN allows the model to achieve optimal clustering while minimizing the overall computational overhead, as our results in Section VII demonstrate.

Formally, let us consider the set of N samples of n features $D \subset \mathbb{R}^{N \times n}$. Let $\psi_{\theta_e}(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be an embedding of a sample x mapped by a fully-connected feed-forward NN

parametrized by $\theta_e \in \mathbb{R}^e$, onto $\mathbb{R}^m$, where m is the target dimension. The embedding of our entire dataset via ψ_{θ_e} can be expressed by $\Psi_{\theta_e} : \mathbb{R}^{N \times n} \rightarrow \mathbb{R}^{N \times m}$. In this setting, we also use the same dimension m for the kernels to be learned by the kernel networks. We now define $\Psi_{\theta_k}(x) : \mathbb{R}^{N \times m} \rightarrow \mathbb{R}^N$ to be our kernel functions with parameters $\theta_k \in \mathbb{R}^k$. For every target class, a new kernel network is generated such that each network learns to represent a unique cluster from the embedding output by the encoder.

In our design, each layer of the encoders implements an activation-like sine function $y_t = w_a \sin(2\pi w_f y_{t-1})$, where y_{t-1} is the output of the previous layer, and w_a and w_f are weight vectors of respective sizes $|w_a| = 1$ and $|w_f| = |y_{t-1}|$, which are learned by the network. The chosen sine activation-like functions enable the networks to learn faster and adapt to complex data structures. For the kernel networks, we use fully-connected neural models, whose outputs are passed through a softmax function $\sigma(y_i) = \exp(y_i) / \sum_{j=1}^N \exp(y_j)$, thereby returning the set of probabilities that a given sample belongs to different clusters. As such, with the kernel networks, we aim to map any embedding from the encoder to single values representing the likelihood estimations that samples belong to their respective clusters. For each sample, this mapping is done through a deep NN while simultaneously the kernel weights defined by these networks, i.e., cluster centers, are computed from the embeddings of all samples.

To train our AC network, we combine two distinct loss functions, i.e., $\mathcal{L} = \mathcal{L}_p + \mathcal{L}_c$, where $\mathcal{L}_p$ is the Mean Squared Error (MSE) between the estimated probabilities of samples belonging to clusters and the ground truth, and $\mathcal{L}_c$ is a contrastive loss that aims to control the distance between the different clusters. The latter is important when the number of clusters grows, hence they become closer to each other and harder to separate. To compute the MSE, we first perform one-hot encoding of the target cluster ids (c_i) as follows:

$$p_i = \begin{cases} 1, & \text{if } i = c_i, \\ 0, & \text{otherwise,} \end{cases}$$

then compute

$$\mathcal{L}_p = \frac{1}{N} \sum_{i=1}^N (p_i - \hat{y}_i)^2,$$

where $\hat{y} = \{\hat{y}_1, \dots, \hat{y}_N\} \in \mathbb{R}^N$ denotes the output probabilities of the model. $\mathcal{L}_c$ is designed to optimize the assignments of the clusters and is defined as

$$\mathcal{L}_c = Y \cdot d_S + (1 - Y) \cdot \max(0, \delta - d_D),$$

where d_S is the distance between all pairs of similar points and d_D is the distance between all pairs of dissimilar points, Y is a binary label indicating whether the pairs are similar (i.e., 1 if they should be deemed similar, 0 otherwise), and $\delta > 0$ is a margin defining the radius around the embedding space of a sample, so that dissimilar pairs only contribute to the loss if $d_D \leq \delta$. The training of the neural model combining the encoders and kernel networks is then performed end-to-end by running back-propagation over a suitable number of iterations.

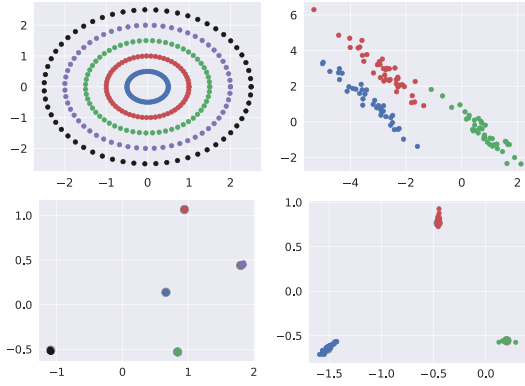


Fig. 3. Illustration of embeddings in a two-dimensional kernel space learned with our AC method: sample datasets (above) and their representations along with cluster centers (below).

By this approach, the embeddings learned by our AC algorithm are easily separable even by the simplest classifiers, as exemplified in Fig. 3. This indicates that the AC network automatically discovers optimal kernels to be learned, which constitutes one of its main advantages as compared to existing clustering methods. Additionally, using cluster centers provided by our approach as features given to classifiers inherently enhances privacy, since all members of the same class would possess the same feature values. In our experiments, we use the same NN structure and the same hyper-parameters. Only the number of sub-networks changes, as this is task-specific and corresponds to the number of output classes.

VI. IMPLEMENTATION

We implement ACID in Python 3.7 using the PyTorch [36] and Scikit-Learn [37] libraries. For our AC algorithm, we employ a set of encoders that are fully-connected NNs with 3 hidden layers comprising 500, 200, and 50 neurons, respectively. The number of neurons in the output layers is equal to the desired dimensionality of the kernels, which we set to 10 in our experiments. The kernel networks are also fully-connected, with 3 hidden layers of size 100, 50 and 30 neurons, respectively.¹ Inputs are processed in mini-batches of size 256 and we train the model using the Adam optimizer [38] with a learning rate of $1e^{-4}$. We adopt a Random Forest (RF) classifier due to its performance and computational efficiency, using default parameters, except the number of trees, which we set to 200. Our complete NIDS is trained over 100 iterations. To mimic computationally constrained edge devices, we execute all our experiments on a virtual machine running Ubuntu 18.04 LTS, with 4GB RAM, 50GB storage space, and a quad-core Intel Celeron N4100 CPU operating at 1.1 GHz. For the same reason, we perform no parallelization and no specific optimization of our clustering algorithm.

VII. PERFORMANCE EVALUATION

We first demonstrate the performance of our AC algorithm on general clustering tasks using five synthetic datasets,

then evaluate ACID with three publicly-available network intrusion datasets. To completely cover all aspects of the NID task, we perform both binary and multi-label classifications on all datasets. We further use one of these datasets for a performance comparison with state-of-the-art NIDSs. Additionally, we perform a complexity and runtime analysis of our solution to understand its deployability on constrained edge devices.

A. Datasets

1) *Synthetic Datasets*: We generate five different artificial datasets covering a range of scenarios with different levels of complexity, including number of clusters/groups, shape, ambiguity, and distributions. Specifically, we consider

- *Two-Circles*: A binary classification task with samples that fall into concentric circles. This is suitable for testing if an algorithm can learn complex non-linear manifolds.
- *Five-Circles*: A multi-label classification problem with samples that fall into concentric circles. Similarly to the Two-circles dataset, this is also suitable for testing if an algorithm can learn complex non-linear manifolds.
- *Two-Moons*: A binary classification problem consisting of samples falling into two interleaved half-circles. This dataset is suitable for testing if an algorithm can learn non-linear and intertwined class boundaries.
- *Blobs*: Groups of data-points with Gaussian distributions, which are suitable for assessing the ability of algorithms to solve linear classification problems.
- *Sine/Cosine*: The samples consist of sine and cosine data points. This dataset is suitable for testing if an algorithm can learn complex, non-linear, and intertwined class boundaries.

With these, we are able to compare the performance of our clustering approach against three popular clustering methods, and ascertaining its universal learning abilities. For visualization purposes, each of these synthetic datasets consists of samples of 2-dimensional data points, whose features correspond to their Cartesian coordinates, and each sample is assigned a label corresponding to its cluster ID.

2) *Intrusion Detection Datasets*: To showcase the performance of our NIDS, we use three datasets that capture a total of 40 types of network attacks collected over a span of 20 years, namely KDD Cup'99 [17], ISCX-IDS 2012 [18], and CSE-CIC-IDS 2018 [19]. The KDD Cup'99 dataset was produced by MIT Lincoln Labs in a LAN operated similarly to an Air Force environment over the course of 9 weeks, during which raw TCP data was collected [39]. The CSE-CIC-IDS 2018 dataset is the result of a controlled attacks campaign run by the Canadian Institute for Cybersecurity using 50 machines that targeted a victim organization with 5 departments, involving 420 machines and 30 servers [40].

For comparison purposes, we evaluate all the benchmarks considered on a random subset of the ISCX-IDS 2012 dataset, which was released by the University of New Brunswick and consists of seven days of raw network data, including benign and four types of malicious network traffic, namely BruteForce SSH, DDoS, HttpDoS, and Infiltration [41].

¹The source code of our implementation is available at <https://github.com/Mobile-Intelligence-Lab/ACID>.

TABLE I
ISCX-IDS 2012 DATASET AFTER PREPROCESSING

Category	Count	Percentage	Training	Validation	Testing
Benign	10,000	28.28%	7,040	296	2,664
BFSSH	7,042	19.92%	4,960	208	1,874
DDoS	4,963	14.03%	3,476	148	1,339
HttpDoS	3,427	9.70%	2,431	99	897
Infiltration	9,925	28.07%	6,903	302	2,720
Total	35,357	100%	24,810	1,054	9,493

B. Preprocessing

Based on the features extracted from raw packets (including fields in the packet headers and statistical attributes), we generate bidirectional flows, thereby incorporating more temporal information than what can be observed from individual packets (e.g., the time interval between two sequential packets). The first packet of each flow determines its direction, i.e., forward (source to destination) or backward (destination to source).

To obtain relatively balanced datasets for benchmarking, we randomly select a predefined number of malicious and benign traffic samples. The preprocessed dataset is then divided into a training, validation, and testing sets using a 70/10/20 split ratio. As an example, we illustrate the preprocessing results obtained for the ISCX-IDS 2012 dataset in Table I.

C. Evaluation Metrics

To measure the performance of our NIDS, we use the held-out testing set to compute confusion matrices, based on which we calculate the number of True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) inferences. With these, we derive a number of metrics that allow us to assess the quality of the classification results of ACID and those produced by the benchmarks considered, namely:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN};$$

$$\text{Precision} = \frac{TP}{TP + FP}; \quad \text{Recall} = \frac{TP}{TP + FN};$$

$$\text{FAR} = \frac{FP}{FP + TN}; \quad \text{F}_1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

In the above, FAR is the false alarm rate.

D. General Clustering Results

We first compare the clustering performance of our AC network against that of Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [42], Spectral Clustering [43], and k-Means [44], which are widely-used clustering approaches. These methods were selected as benchmarks due to their prevalent use in the literature, enduring relevance, and diverse mechanisms for capturing different types of cluster structures. For completeness, two of the most recently proposed clustering algorithms were also included for comparison, namely HDBSCAN [46] and Mean Shift [45]. To obtain a fair comparison with these methods, we specifically tune the parameters of each to optimize their performance.

Clustering Algorithm	Datasets (Purity Score)				
	2 Circles	5 Circles	2 Moons	Blobs	Sine / Cosine
Ground Truth					
k-Means					
	(0.50)	(0.40)	(0.74)	(0.78)	(0.50)
Mean Shift					
	(0.52)	(0.20)	(0.80)	(0.84)	(0.50)
Spectral Clustering					
	(1.00)	(0.60)	(1.00)	(0.93)	(0.51)
DBSCAN					
	(1.00)	(1.00)	(1.00)	(0.96)	(0.60)
HDBSCAN					
	(1.00)	(1.00)	(1.00)	(0.64)	(0.60)
AC (Ours)					
	(1.00)	(1.00)	(1.00)	(1.00)	(1.00)

Fig. 4. Comparison of clustering results of the proposed AC method and five popular benchmarks on two-circles, five-circles, two-moons, blobs, and sine/cosine datasets. The Purity The Purity Score [47] metric is used to evaluate the quality of all clustering results (shown below each plot).

As observed in Fig. 4, Spectral Clustering obtains optimal results with the two-circles and the two-moons datasets, while DBSCAN performs very well on three out of the five datasets, i.e., two-circles, five-circles, and two-moons. Both approaches partially misclassify the blobs and consistently fail to cluster correctly the sine/cosine dataset. The k-Means clustering algorithm, however, systematically fails on all these tasks.

In contrast, our AC approach flawlessly clusters the data points in all the datasets considered, regardless of shape, distribution, or complexity (Fig. 4, rightmost column). This demonstrates the key advantage of using kernel networks to identify cluster centers and augmenting the feature set with information about these centers in view of classification.

E. Network Intrusion Detection Results

Next, we evaluate the performance of our complete NIDS. Recall that ACID extrapolates meaningful low-dimensional representations from header and statistical features extracted from raw network traffic data. Using these automatically learned features, we determine different cluster centers and use them to extend the header and statistical attributes. With these additional features, we expect our classifier to be more accurate and easily distinguish even the most similar patterns.

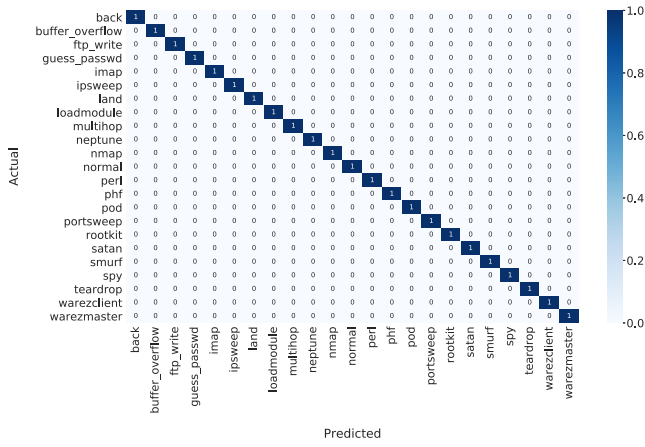


Fig. 5. Normalized confusion matrix for multi-label classification using ACID on the KDD Cup'99 dataset.

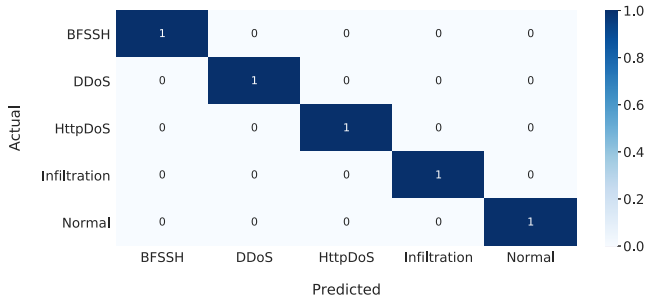


Fig. 6. Normalized confusion matrix for multi-label classification using ACID on the ISCX-IDS 2012 dataset.

To verify our hypothesis, we perform binary and multi-label classification on the three real-world datasets mentioned above, where we distinguish benign/malicious traffic flows, and respectively identify the type of every single traffic flow.

We also experimentally compare our approach with recent NIDS designs based on neural networks. In particular, we perform classification using DAGMM [14], which employs Gaussian Mixture Models (GMMs) to learn low-dimensional embeddings of complex data structures while avoiding undesired local optima; N-BaIoT [15], which relies on Auto-Encoders (AEs) to discriminate IoT traffic; Deep NNs [16], which offer provably high performance despite using a simple architecture; CNN-BiLSTM [48] combining a CNN and a Bi-directional LSTM to obtain high detection rates while minimizing false positive rates; and TR-IDS [25], which exploits payload contents to enhance NID performance.

In Figs. 5, 6, and 7, we provide normalized confusion matrices obtained with ACID, demonstrating its performance on the KDD Cup'99, ISCX-IDS 2012, and CSE-CIC-IDS 2018 datasets, respectively. Observe that our approach produces perfect results in the multi-label classification task, even where some network traffic flows may be very similar, e.g., Distributed Denial-of-Service (DDoS) attacks and high volume benign traffic. ACID correctly classifies 100% of the traffic flows when using both kernel and payload features. Accuracy degrades only marginally when payload features are not employed for classification, specifically 99.41% accuracy is attained on the CSE-CIC-IDS 2018 dataset in this scenario.

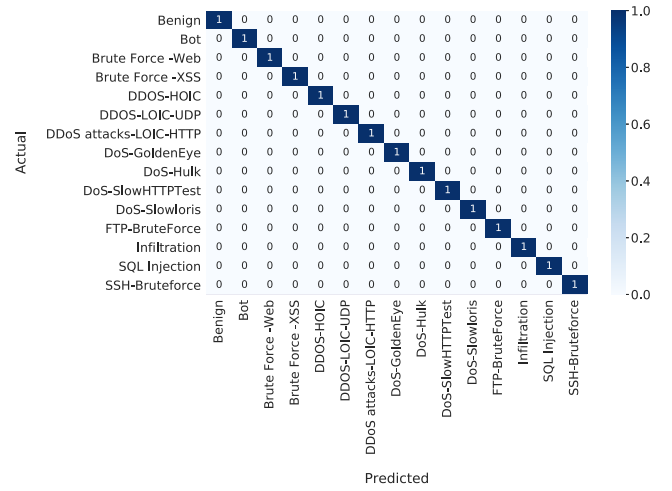


Fig. 7. Normalized confusion matrix for multi-label classification using ACID on the CSE-CIC-IDS 2018 dataset.

TABLE II
PERFORMANCE SUMMARY OF ACID ON THE KDD CUP'99, THE ISCIX-IDS 2012, AND THE CSE-CIC-IDS2018 DATASETS FOR THE MULTI-LABEL CLASSIFICATION TASK

Metric	Using Payload	Accuracy	FAR	F ₁	Samples
Dataset		(%)	(%)	(%)	
KDD CUP'99	No	99.92	0.08	99.92	39,159
ISCX-IDS 2012		99.63	0.39	99.59	9,493
CSE-CIC-IDS 2018		99.41	0.61	99.40	130,295
KDD CUP'99	Yes	100.0	0.00	100.0	39,159
ISCX-IDS 2012		100.0	0.00	100.0	9,493
CSE-CIC-IDS 2018		100.0	0.00	100.0	130,295

We also compute the accuracy, precision, recall, F₁-score, and FAR for all datasets. The results confirm that ACID attains 100% accuracy, 0% FAR, and 100% F₁-score, when performing both binary and multi-label classification, irrespective of the number of classes. Multi-label classification results are summarized in Table II. These remarkable performance can be attributed to the manner in which our approach acts on the data, which is akin to a two-stage classification process, where the first stage corresponds to classifying network traffic via clustering, and the second corrects the misclassified samples through a further classifier. These results also confirm that our learned features consist of transferable knowledge across all samples in the respective datasets.

We further juxtapose ACID with the benchmark NIDSs considered, when classifying traffic in the ISCX-IDS 2012 dataset. We limit this comparison to binary classification, which is the intended goal of most of these methods. The results obtained are shown in Table III, which reveals that ACID outperforms existing solutions by up to 47% in terms of F₁-score. By combining a Text-CNN and RF, TR-IDS attains very good performance on the binary classification task, but unlike ACID, it struggles to discriminate malicious traffic flows of different types that are superficially similar. Specifically, in multi-label classification, TR-IDS misclassifies ~1% of DDoS and ~1% Infiltration as benign flows, while flagging more

TABLE III
COMPARISON OF ACID WITH EXISTING METHODS ON BINARY CLASSIFICATION WITH ISCX-IDS 2012 DATASET

Approach	Payload-based Features	Accuracy (%)	FAR (%)	F ₁ (%)
DAGMM [14]	No	62.91	30.65	53.07
N-BaIoT [15]	No	89.19	10.80	89.19
Deep NN [16]	No	88.14	7.41	70.35
CNN-BiLSTM [48]	No	98.43	1.57	98.42
ACID (ours)	No	99.78	0.23	99.44
TR-IDS [25]	Yes	98.88	1.12	98.87
ACID (ours)	Yes	100.0	0.00	100.0

than 1% of benign flows as attacks. In practice, this would not only lead to manual inspection of large numbers of flows, but allow attacks with dramatic consequences (e.g., Infiltration) to succeed, and potentially block traffic with commercial value.

The generation of network traces for these NID datasets taking place in controlled testbed environments, their traffic patterns exhibit much lower variability than can be expected in real network environments. Despite the remarkable classification performance of ACID within these testbeds, achieved through a thorough analysis of traffic dynamics from packet headers and patterns found in payloads, further research would be warranted to measure the true impact of ACID in real-world deployments. Nevertheless, the substantial improvements demonstrated over the benchmark methods serve as compelling evidence that our proposed architecture excels in capturing the intricate patterns and dynamics that underlie network traffic flows.

F. Complexity

Having demonstrated exceptional NID performance, we now study the complexity of our ACID approach, both from computational and runtime perspectives. To this end, we count the number of parameters of our neural model and the number the floating point operations (FLOP) performed per inference. We also measure the inference time for a *single sample* and a batch of *128 samples*, respectively, accounting for all the processing undergone by packet through ACID's complete pipeline. The results obtained are reported in Table IV, where we also assess the additional complexity incurred when using payload features. Given that we use an edge device emulation set-up (described in Section VI) and inference times can be as low as 80ms, we conclude that it is feasible to deploy our ACID system on constrained edge devices for intrusion detection purposes. Further, increasing the batch size to 128 reduces the runtime per sample by 100×. We also note that payload features incur ~2× higher execution time per single flow inference, while they can prove orders of magnitudes more costly when working with proportionally larger batches, which needs to be accounted for when deploying at the network edge.

VIII. ABLATION STUDY

A. Sensitivity Analysis

We conclude with a sensitivity analysis of our AC algorithm wrt. (i) the dimensionality of the learned representations, and (ii) the importance of cluster centers in the final classification.

TABLE IV
COMPUTATIONAL COMPLEXITY OF OUR CLUSTERING APPROACH TO NIDS, WITH AND WITHOUT PAYLOAD FEATURES. EXPERIMENTS ON AN EMULATED CONSTRAINED DEVICE AS DEFINED IN SECTION VI

Payload Features	Number of Parameters	Batch size	Model Complexity (MFLOP)	Execution Time (seconds)
No	789,855	1	1.49	0.08 ± 0.01
		128	191.68	0.10 ± 0.02
Yes	942,460	1	25.71	0.19 ± 0.04
		128	3291.43	18.59 ± 0.74

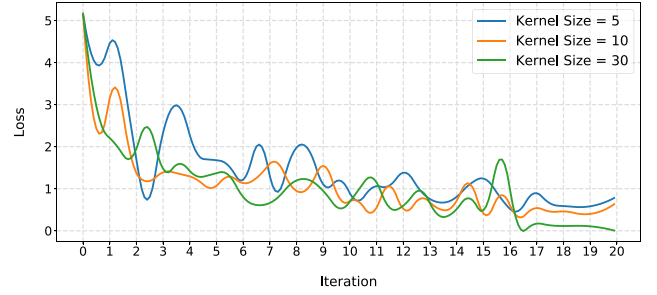


Fig. 8. Evolution of loss function values for different kernel sizes when training AC on the ISCX-IDS 2012 dataset.

1) *Kernel Size*: In the first experiment, without changing any other parameter of our model, we vary the kernel size and evaluate its impact on the quality of the clustering. Specifically, we examine the loss curve during training when the kernel size is respectively 5, 10, and 30. The results are shown in Fig. 8.

Observe that the impact of the dimension of the kernels is relatively negligible, as our AC approach converges rapidly to small loss values that lead to efficient separation of data samples into different clusters. This observation is particularly valuable when considering deploying our NIDS on constrained devices. For this reason, working with 10 as the NIDS kernel size, as we did in all experiments reported in this paper, is reasonable. Also note that training our clustering method for only 20 iterations is sufficient to obtain state-of-the-art performance and reduce the False Alarm Rate (FAR) to zero.

2) *Feature Importance*: To better understand the impact of different features on the classification results, we analyze their degree of contribution to this process. Since we use the RF classifier in the final stage of ACID, its implementation in the Scikit-Learn library directly provides the importance weight of each feature, thereby making it easy to rank all features according to their importance score. From these aggregated features, we select the top-15 ones according to their relative importance to the decision process and plot them in Fig. 9. To appreciate the importance of the cluster centers relative to all header and statistical features extracted, as well as payload based features, we extract 50 features from the payloads, using two modern Natural Language Processing (NLP) techniques (word embedding and Text-CNN), as also performed in [25]. Recall that these payload features can help to detect malicious contents, such as those seen with payload-based attacks, i.e., SQL injection, cross-site scripting (XSS), and shell-code.

Finally, we perform the same experiment excluding the payload features and observe that the cluster centers extracted

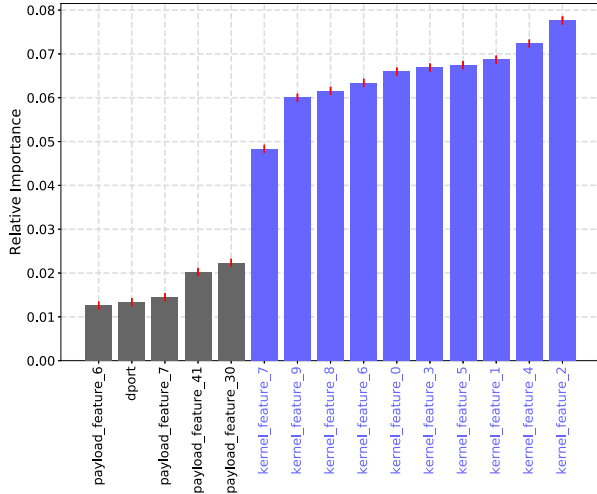


Fig. 9. 15 most important features in the classification process on the ISCX-IDS 2012 dataset. 50 features extracted from the payloads in total.

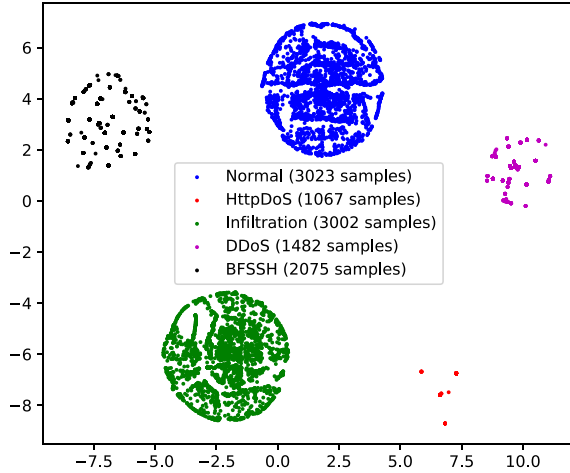


Fig. 10. (t-SNE) 2-D projections of clusters obtained by our AC approach for multi-label classification of ISCX-IDS 2012.

from our clustering algorithm significantly outweigh all other features during the decision process. More specifically, the cluster centers contribute to the decision process by 5.77% to 9.03% (almost 2 to 3.6 times more than the most important of all header and statistical features combined). Furthermore, the clusters obtained by our AC approach to NID provide perfectly separable representations of our data points for all network traffic categories, which is confirmed by the t-SNE representation [57] of the clusters shown in Fig. 10. Indeed, reducing the dimension of embedded representations obtained by our model to 2 through this method reveals that the clusters corresponding to all types of attacks and benign traffic are clearly distinguishable to the clustering algorithm.

B. Impact of Classification Algorithm on ACID's Performance

In this section, we examine the impact of different classification algorithms as the final layer of our ACID framework. This evaluation is crucial for understanding how different classifiers influence ACID's overall performance in detecting

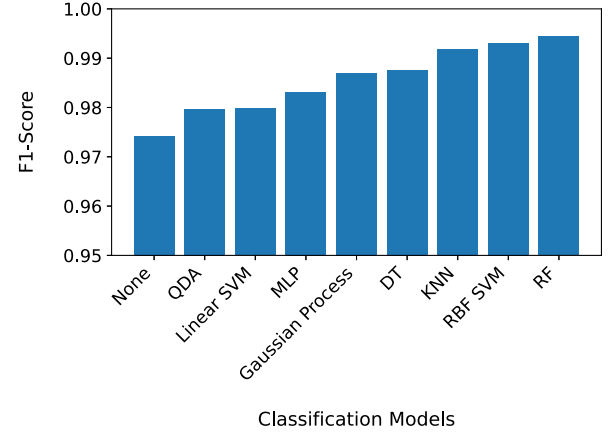


Fig. 11. Performance of ACID with different classifiers: No classification layer, Quadratic Discriminant Analysis [49], Linear SVM [50], MLP, Gaussian Process [51], Decision Tree [52], kNN [53], RBF SVM [54], and RF [35].

TABLE V
PERFORMANCE EVALUATION OF CNN-BiLSTM AND ACID (WITHOUT PAYLOAD-BASED FEATURES) AGAINST RECENT NETWORK INTRUSION DETECTION DATASETS

Dataset	Approach	Accuracy (%)	FAR (%)	F ₁ (%)
OPC UA	CNN-BiLSTM	97.96	2.04	97.98
	ACID (ours)	99.85	0.71	99.33
DoHBrw	CNN-BiLSTM	99.02	0.98	99.02
	ACID (ours)	99.98	0.02	99.98

and categorizing network intrusions. The results of this exploration, shown in Fig. 11, further ascertain the adaptability and versatility of ACID, establishing it as an effective and flexible solution for network security applications.

C. Evaluations on Recent NID Datasets

In this section, we assess the robustness of our proposed framework against more recent network traffic datasets. Specifically, we extend our evaluation to incorporate the OPC UA [55] and the CIRA-CIC-DoHBrw-2020 [56] datasets. This expansion of our evaluation scope highlights the adaptability and efficacy of ACID in the face of contemporary threats and affirms the practical applicability and relevance of ACID in evolving network landscapes. Table V summarizes the results of our evaluations.

IX. PRACTICAL CHALLENGES

For NIDS to be practically viable, they need to maintain their performance over time, be compatible with different network architectures, and handle network traffic distributions that are different than those observed during training. In this section, we evaluate the performance of our proposed framework in light of two of the most challenging issues: data corruption (modelled from random distributions) and evolution of network traffic (categorized as concept and data drifts). To tackle these practical challenges, we simulate both conditions using the CIC-IDS-2017 dataset [41], [58] with entries grouped into Benign traffic and 5 attack categories,

namely: Bot, Brute-Force, DoS/DDoS, Infiltration, and Web Attack. For all experiments, we split the dataset into a training and testing set using a 70/30 ratio, and make use of header and statistical features alone to exclude performance gains introduced by payload features. The experiments performed highlight the advantages of using our proposed framework compared to traditional NN architectures, where we use as baseline a 5-layer MLP consisting of: feed-forward, ReLU activation, batch-normalization, and dropout layers for each block [16].

A. Robustness to Data Corruption

To verify the comprehensive performance of ACID in classifying network traffic when deployed in noisy environments, we simulate corrupted features from different distributions. For each distribution function we make a comparison between our proposed architecture and that of the classical NN. As evaluation metric, we measure the accuracy gain/loss incurred by the noise. Formally, let $a^d(m)$ denote the classification accuracy of a model for any given probability distribution d and any given noise magnitude $m \in [0..1]$. The Noise-Induced Relative Error (NIRE) in percentage is thus given by:

$$NIRE(m, d) = \frac{a^d(m) - a^d(0)}{a^d(0)} \times 100\%,$$

where $a^d(0)$ is the accuracy of the models without any noise (i.e., 0% noise magnitude). We generate corrupted network traffic features to evaluate the NIDS architectures by applying perturbations drawn from 5 different random distributions. Namely:

- *Uniform*: generates random values with equal probability over a given range using a probability density function defined as:

$$f(x) = \frac{1}{b - a},$$

where a and $(b - a)$ are the location and scale parameters, respectively.

- *Bernoulli*: generates random values consisting of one of two possible outcomes ($x = 0$ and $x = 1$) defined by the probability density function:

$$f(x) = p^x(1 - p)^{1-x},$$

where p is the probability of having $x = 1$ and $1 - p$ the probability of having $x = 0$.

- *Normal*: produces values symmetrically centered around a given mean (μ) with a standard deviation (σ), defined by the following probability density function:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}.$$

- *Log-Normal*: describes random variables whose logarithms follow a normal distribution. For any given mean (μ) and standard deviation (σ), its probability density function is given by:

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} e^{-(\ln(x)-\mu)^2/(2\sigma^2)}.$$

- *Exponential*: describing the inter-arrival times in a Poisson process, it is defined using a probability density function:

$$f(x) = \lambda e^{-\lambda x},$$

for a constant average rate λ .

Each of these distributions is parametrized to produce noises of specific magnitudes employed to perturb time-based features of network traffic flows, such as: packet inter-arrival time, active-idle time, and number/size of packets transferred. In Fig. 12 we report the performance comparison as NIRE, examining a range of noise magnitudes (0–5%, 10%, 15%, 20%, 25%, 50%, and 100%). Negative values indicate performance degradation, while positive ones reflect potential performance improvements.

The results obtained show negligible to no degradation of ACID's performance for perturbation magnitudes of up to 25%, regardless of the nature of the perturbation (i.e., distribution from which the perturbation is drawn). Further, we note that when using our proposed framework, low-magnitude perturbations have a tendency to improve the classification performance (up to 7.5% with the experiment performed). These results confirm our intuition that, compared to traditional NN architectures, our ACID framework inherently provides a high degree of tolerance to random perturbations.

B. Continual Learning

Despite significant performance boosts offered by NNs for the intrusion detection task, the ever-evolving nature of network traffic in real-world environments presents a major challenge. In fact, with the increasing heterogeneity and complexity of network environments, most NIDS are faced with high volumes of traffic continuously changing in nature, either due to the emergence of new types of traffic or by changes in the underlying structure of known types (e.g., the codecs employed by media streaming services). While defense systems usually perform well in static environments, this dynamicity degrades their performance over time. This is mainly due to the static and fixed structure of traditional DNNs, which limits their learning capability. Further, with new types of traffic emerging, previously learned knowledge may be erased by training the models on the newly encountered data distributions (known as catastrophic forgetting [59], [60], [61]).

To overcome this issue DNNs traditionally incorporate new knowledge by retraining with both the new data and all previous training data, which requires massive amounts of computational and memory resources as more data is encountered. This is however impractical, especially in the context of IoT settings where devices are characterized by limited processing, memory, and networking capabilities [62]. As such, Continual Learning solutions have been proposed, whereby ML models are trained only using newly acquired data (also known as Incremental Learning), which on top of solving this problem, also improves speed and memory efficiency.

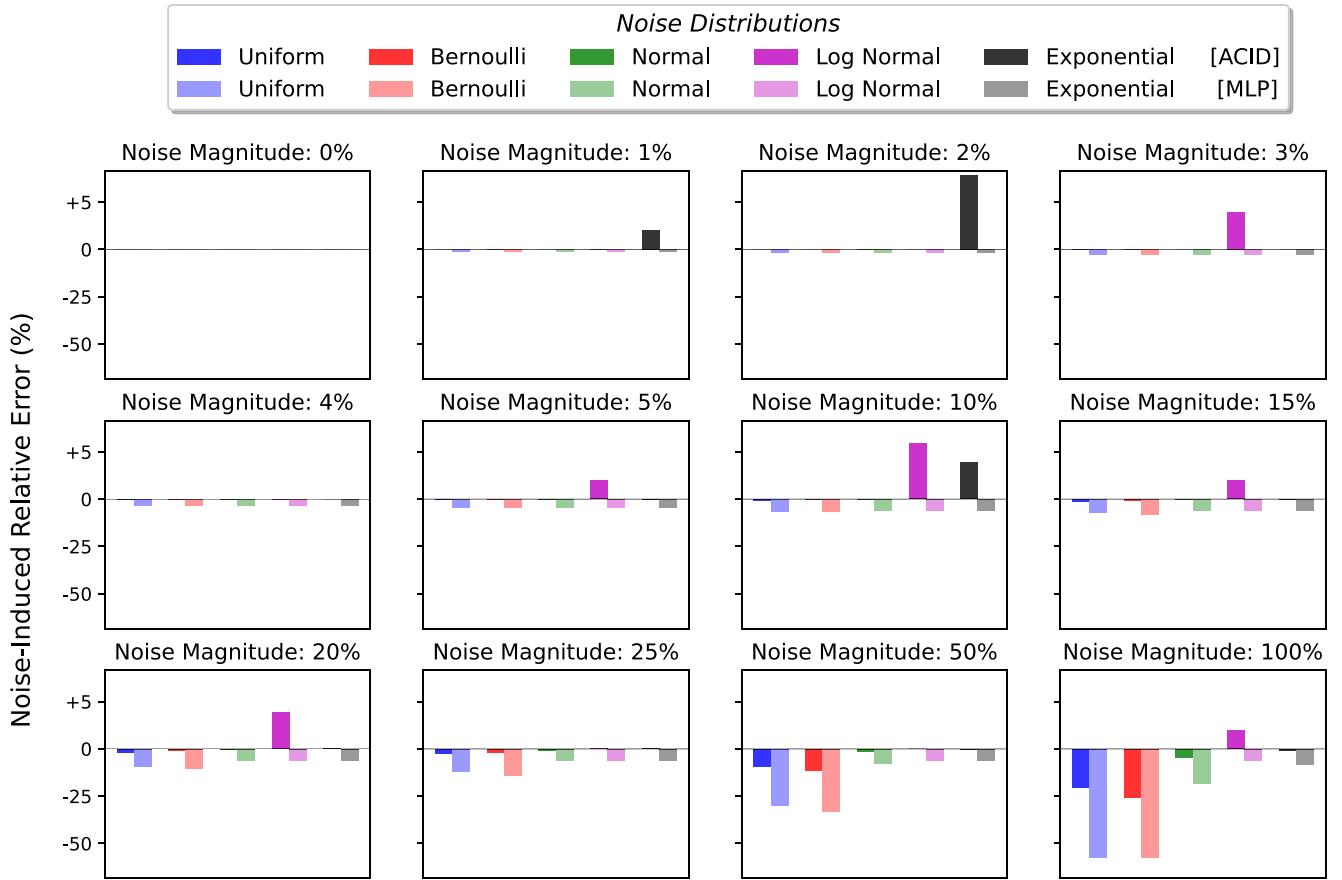


Fig. 12. Performance comparison of ACID and traditional NN architectures in terms of robustness to random noise. Perturbations generated for increasing magnitudes of noise and drawn from different distributions. Negative values indicate performance degradation; positive values indicate improvements.

In what follows, we compare ACID to a traditional NN architecture and show that ACID, with its task-oriented sub-nets, inherently incorporates the notion of Continual Learning in its learning process, by creating new sub-nets for each new task encountered. This allows our proposed framework to preserve its performance over time by learning and adapting its structure when new training data are available, continually expanding its knowledge of previously learned tasks without requiring to retrain models from scratch. In other words, the architectural design of our proposed Adaptive Clustering networks overcomes the need of storing any previous training datasets while maintaining the stability of the NIDS.

1) *Class-Incremental Learning*: We first consider the Class-Incremental Learning (CIL) problem, where each task consists of learning from data collected for a previously unseen class of network traffic (e.g., type of attacks). In this scenario, the CIC-IDS-2017 dataset is divided into a sequence of tasks, each task associated with a separate training session where the NIDS only has access to the data of the current task. We aim to verify ACID's intrinsic ability to prevent catastrophic forgetting while (i) maintaining its performance on all previously learned tasks and (ii) avoiding the problem of intransigence (failure to adapt to new tasks). Our experiment is designed to observe the performance of the NIDS at each stage of the CIL learning process.

We undertake a performance comparison of our approach with an MLP (representing a traditional DNN), and show that where traditional NN structures are biased towards recently-learned tasks (task-recency bias), leading to catastrophic forgetting. Similar to ACID, this MLP, consisting of six feed-forward layers (with non-linear activations), is trained and evaluated using a traditional Class-Incremental Learning approach, where both models are sequentially exposed to new classes of network traffic over time. At each stage, the networks are trained using samples from a new class, and evaluated on test samples from all previously learned classes.

ACID maintains consistent performance across stages of the learning process (see Fig. 13). While not guaranteed, we note that despite the precipitous degeneration effect observed with classical DNN architectures, they can partially recover their performance on a previous task if the previous and the new tasks share some underlying attributes. Our proposed architecture on the other hand overcomes any degeneration effect resulting from the introduction of new tasks, since the training and classification of any given task's samples is handled by a dedicated sub-net.

2) *Concept Drift Adaptation*: Having confirmed ACID's ability to learn new tasks based on newly available training sets alone, and without degrading its performance on previously learned tasks, we now assess the classification accuracy under the effect of conceptual drifts, where statistical properties

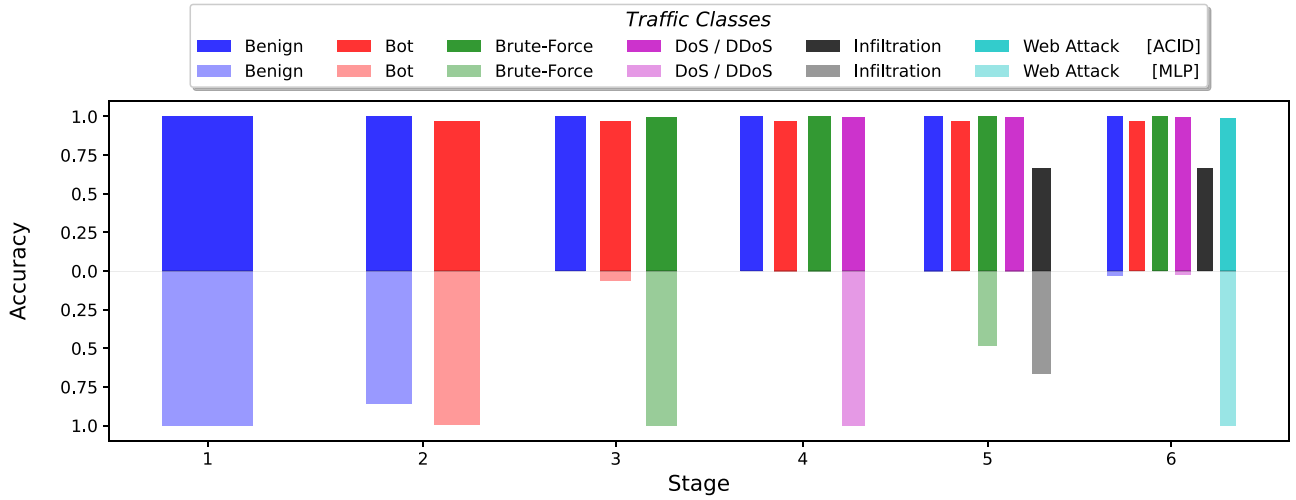


Fig. 13. Performance comparison of ACID (top) and MLP (bottom) at different stages of a Class-Incremental Learning process, by which different attack types are observed sequentially.

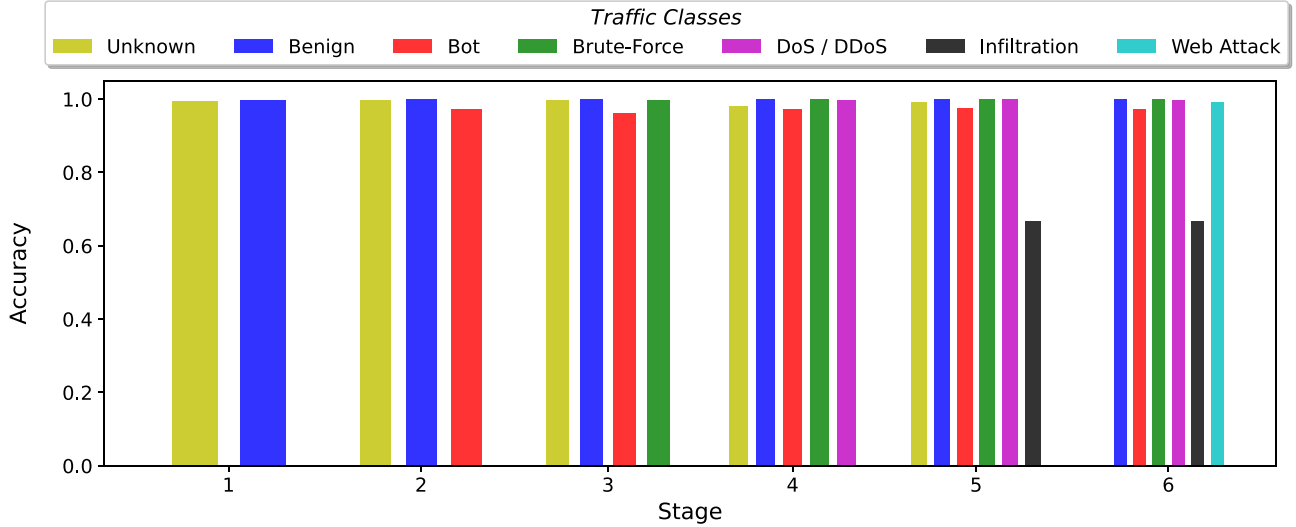


Fig. 14. Adaptive performance of ACID when presented with conceptual drifts. At each Class-Incremental Learning stage, the “Unknown” category represents all classes of traffic that have yet to be learned by new sub-nets.

(relationships between input and output data) of our NIDS change over time. To simulate this notion, we extend the previous experiment (presented in Section IX-B1) by attempting to classify any network traffic not yet learned by ACID as “unknown”. This change of distribution observed for this class of outliers, as we learn new types of traffic at each stage of the CIL process, thereby characterizes our concept drifts. The results depicted in Fig. 14 show that throughout the learning process, the definition of “unknown” traffic is continually expended by our solution, thereby enabling the extension of its knowledge base with a new sub-net for each new type of traffic, while updating its knowledge of the “unknown” types.

X. CONCLUSION AND FUTURE DIRECTIONS

In this paper, we introduced a novel approach to Network Intrusion Detection (NID) based on an Adaptive Clustering

(AC) neural network that achieves exemplary performance on three different datasets, in both binary and multi-label traffic classification tasks. Our design hinges on multiple kernel networks to learn optimal embeddings of data samples, thereby acquiring the ability to easily distinguish different types of network traffic. Through extensive experiments, we have proved the superiority of our clustering method over existing alternatives, and made the case for a lightweight and effective Network Intrusion Detection System (NIDS) that can be deployed on devices with limited computational resources, thereby strengthening defenses at the network edge. We have subsequently studied the viability of our approach in practical settings by highlighting its robustness to noise and its intrinsic support for continual learning.

As future work, we will extend our Adaptive Clustering framework to unsupervised classification tasks, virtually eliminating the need for high-quality labelled data.

REFERENCES

- [1] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [2] *The IoT Business Index: A Steep Change in Adoption*, Economist Intelligence Unit, London, U.K., Feb. 2020.
- [3] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, "Demystifying IoT security: An exhaustive survey on iot vulnerabilities and a first empirical look on internet-scale IoT exploitations," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2702–2733, 3rd Quart., 2019.
- [4] *2019 Internet Crime Report*, Federal Bureau of Investigat. (FBI), Washington, DC, USA, Feb. 2020.
- [5] Z. Inayat, A. Gani, N. B. Anuar, M. K. Khan, and S. Anwar, "Intrusion response systems: Foundations, design, and challenges," *J. Netw. Comput. Appl.*, vol. 62, pp. 53–74, Feb. 2016.
- [6] (Cisco, San Jose, CA, USA). "Snort." Accessed: Sep. 27, 2023. [Online]. Available: <https://talosintelligence.com/snort>
- [7] "Zeek." Accessed: Sep. 27, 2023. [Online]. Available: <https://zeek.org/>
- [8] "Suricata." Accessed: Dec. 26, 2023. [Online]. Available: <https://suricata.io>
- [9] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Appl. Sci.*, vol. 9, no. 20, p. 4396, 2019.
- [10] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2nd Quart., 2016.
- [11] A. B. Nassif, I. Shahin, I. Attili, M. Azzeh, and K. Shaalan, "Speech recognition using deep neural networks: A systematic review," *IEEE Access*, vol. 7, pp. 19143–19165, 2019.
- [12] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. E. Alsaadi, "A survey of deep neural network architectures and their applications," *Neurocomputing*, vol. 234, pp. 11–26, Apr. 2017.
- [13] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart., 2019.
- [14] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. ICLR*, Mar. 2018, pp. 1–19.
- [15] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul.–Sep. 2018.
- [16] R. Vinayakumar, M. Alazab, K. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.
- [17] "KDD cup'99." Accessed: Sep. 27, 2023. [Online]. Available: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [18] "ISCX-IDS 2012." Accessed: Sep. 27, 2023. [Online]. Available: <https://www.unb.ca/cic/datasets/ids.htm>
- [19] "CSE-CIC-IDS 2018." Accessed: Sep. 27, 2023. [Online]. Available: <https://registry.opendata.aws/cse-cic-ids2018>
- [20] Y. Yu, J. Long, and Z. Cai, "Session-based network intrusion detection using a deep learning architecture," in *Proc. Int. Conf. Model. Decis. Artif. Intell.*, Sep. 2017, pp. 144–155.
- [21] Y. Yu, J. Long, and Z. Cai, "Network intrusion detection through stacking dilated convolutional autoencoders," *Security Commun. Netw.*, vol. 2017 pp. 1–10, Nov. 2017.
- [22] M. Yousefi-Azar, V. Varadharajan, L. Hamey, and U. Tupakula, "Autoencoder-based feature learning for cyber security applications," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, May 2017, pp. 3854–3861.
- [23] Z. Tan, A. Jamdagni, X. He, P. Nanda, R. Liu, and J. Hu, "Detection of denial-of-service attacks based on computer vision techniques," *IEEE Trans. Comput.*, vol. 64, no. 9, pp. 2519–2533, Sep. 2015.
- [24] W. Wang et al., "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2017.
- [25] E. Min, J. Long, Q. Liu, J. Cui, and W. Chen, "TR-IDS: Anomaly-based intrusion detection through text-convolutional neural network and random forest," *Security Commun. Netw.*, vol. 2018, pp. 1–9, Jul. 2018.
- [26] S. Nejatian, H. Parvin, and E. Faraji, "Using sub-sampling and ensemble clustering techniques to improve performance of imbalanced classification," *Neurocomputing*, vol. 276, pp. 55–66, Feb. 2018.
- [27] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, no. 1, pp. 321–357, 2002.
- [28] M. Jianliang, S. Haikun, and B. Ling, "The application on intrusion detection based on k-means cluster algorithm," in *Proc. IEEE Inf. Technol. Appl.*, vol. 1, May 2009, pp. 150–152.
- [29] Y. Guan, A. Ghorbani, and N. Belacel, "Y-means: A clustering method for intrusion detection," in *Proc. IEEE Can. Conf. Elect. Comput. Eng. Toward Caring Humane Technol.*, vol. 2, Jun. 2003, pp. 1083–1086.
- [30] Z. Mingqiang, H. Hui, and W. Qian, "A graph-based clustering algorithm for anomaly intrusion detection," in *Proc. IEEE Comput. Sci. Educ.*, Jul. 2012, pp. 1311–1314.
- [31] Z. Li, Y. Li, and L. Xu, "Anomaly intrusion detection method based on k-means clustering algorithm with particle swarm optimization," in *Proc. IEEE Inf. Technol. Comput. Eng. Manag. Sci.*, vol. 2, Sep. 2011, pp. 157–161.
- [32] F. Hachmi and M. Limam, "A two-stage technique to improve intrusion detection systems based on data mining algorithms," in *Proc. IEEE Int. Conf. Model. Simulat. Appl. Optim.*, Apr. 2013, pp. 1–6.
- [33] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. ICLR Workshop*, 2013, pp. 1–12.
- [34] T. He, W. Huang, Y. Qiao, and J. Yao, "Text-attentional convolutional neural network for scene text detection," *IEEE Trans. Image Process.*, vol. 25, no. 6, pp. 2529–2541, Jun. 2016.
- [35] L. Breiman, "random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [36] "PyTorch." Accessed: Sep. 27, 2023. [Online]. Available: <https://pytorch.org/>
- [37] "Scikit-learn." Accessed: Sep. 27, 2023. [Online]. Available: <https://scikit-learn.org/stable/>
- [38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2014, pp. 1–15.
- [39] J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, "Cost-based modeling and evaluation for data mining with application to fraud and intrusion detection," in *Proc. Results JAM Project Salvatore*, 2000, pp. 1–15.
- [40] A. Shiravi, H. Shiravi, M. Tavallae, and A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Comput. Security*, vol. 31, no. 3, p. 357–374, May 2012.
- [41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.
- [42] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, vol. 96, 1996, pp. 226–231.
- [43] F. R. Chung and F. C. Graham, *Spectral Graph Theory*. Providence, RI, USA: Amer. Math. Soc., 1997.
- [44] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. Berkeley Symp. Math. Statist. Probabil.*, vol. 1, 1967, pp. 281–297.
- [45] D. Comaniciu and P. Meer, "Mean shift: A robust approach toward feature space analysis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 5, pp. 603–619, May 2002.
- [46] L. McInnes, J. Healy, and S. Astels, "Hdbscan: Hierarchical density based clustering," *J. Open Sour. Softw.*, vol. 2, no. 11, p. 205, 2017.
- [47] H. Schütze, C. D. Manning, and P. Raghavan, *Introduction to Information Retrieval*, Cambridge, U.K.: Cambridge Univ. Press, vol. 39, 2008, pp. 234–265.
- [48] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 3rd Int. Conf. Artif. Intell. Pattern Recognit.*, 2020, pp. 223–231.
- [49] T. Hastie, R. Tibshirani, and J. H. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, vol. 2. New York, NY, USA: Springer, 2009, pp. 1–758.
- [50] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, pp. 273–297, Sep. 1995.
- [51] C. E. Rasmussen, and C. K. I. Williams, *Gaussian Processes for Machine Learning*, vol. 1. Cambridge, MA, USA: MIT Press, 2006.
- [52] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, "Classification and regression trees," in *Wadsworth and Brooks*, New York, NY, USA: Routledge, 1984.
- [53] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [54] B. E. Boser, I. M. Guyon, and V. N. Vapnik, "A training algorithm for optimal margin classifiers," in *Proc. 5th Annu. Workshop Comput. Learn. Theory*, 1992, pp. 144–152.
- [55] R. Pinto, *M2M Using OPC UA*, IEEE Dataport, Porto, Portugal, 2020.

- [56] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of doh tunnels using time-series classification of encrypted traffic," in *Proc. IEEE Int. Conf. Depend. Auton. Secure Comput. Int. Conf. Pervasive Intell. Comput. Int. Conf. Cloud Big Data Comput. Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCom/CyberSciTech)*, 2020, pp. 63–70.
- [57] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, Nov. 2008.
- [58] "CSE-CIC-IDS 2017." Accessed: Sep. 27, 2023. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [59] S. Lewandowsky and S.-C. Li, "Catastrophic interference in neural networks: Causes, solutions, and data," in *Interference and Inhibition in Cognition*, F. N. Dempster, C. J. Brainerd, and C. J. Brainerd, Eds. San Diego, CA, USA: Elsevier, 1995, pp. 329–361.
- [60] R. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, pp. 128–135, Apr. 1999.
- [61] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, p. 436, May 2015.
- [62] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019. [Online]. Available: <https://doi.org/10.1109/tkde.2018.2876857>
- [63] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," 2013, *arXiv:1312.6211*.
- [64] J. von Oswald, C. Henning, B. F. Grewe, and J. Sacramento, "Continual learning with hypernetworks," 2019, *arXiv:1906.00695*.
- [65] T. J. Draelos et al., "Neurogenesis deep learning: Extending deep networks to accommodate new classes," in *Proc. IJCNN*, 2017, pp. 526–533.
- [66] A. A. Rusu et al., "Progressive neural networks," 2016, *arXiv:1606.04671*.
- [67] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. ICML*, 2017, pp. 1126–1135.



Alec F. Diallo received the joint integrated master's degree from Mundiapolis University and ESIEE Paris, with a focus on computer science and electrical engineering. He is currently pursuing the Ph.D. degree with The University of Edinburgh. His current research seeks to bridge the gap between the ever-evolving nature of cyber threats and the security and privacy of users' data on networked systems, by using artificial intelligence to build automatic threat detection and counteraction mechanisms.



Paul Patras (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees from the Universidad Carlos III de Madrid. He is an Associate Professor with the School of Informatics, The University of Edinburgh, where he leads the Mobile Intelligence Lab – a multi-disciplinary team that pursues research at the intersection of network engineering and artificial intelligence, to improve the analysis, resilience, and management of next generation mobile systems. He is also a Co-Founder and the CEO of Net AI, a pioneering university spinout specializing in AI-driven network analytics. He was the recipient of a prestigious Chancellor's Fellowship awarded by the University of Edinburgh. He has served on the organizing committee on several conferences and workshops in his field, and advised the ITU-T Focus Group on machine learning for future networks, including 5G.